Bilkent University

Department of Electrical and Electronics Engineering

LAB 4: Arithmetic Logic Unit

Purpose

The main objective of this lab is the creation, simulation, and hardware realization of a 4-bit Arithmetic Logic Unit (ALU). This unit is designed to compute exactly eight distinct mathematical and bitwise operations. These procedures involve basic arithmetic like addition and subtraction, alongside specific shift and logic functions. A core principle of this project relies on a strictly modular, gate-level design methodology. Rather than implementing standard VHDL arithmetic packages, the entire ALU logic is constructed entirely from scratch.

Independent sub-modules, including simple logic gates, single-bit adders, and multiplexers, are developed in VHDL and structurally combined utilizing Vivado's Block Design capabilities. Finally, the lab aims to demonstrate the transition from software simulation to physical hardware performance. After carefully verifying the boolean logic via VHDL testbenches, the architecture is synthesized and flashed onto an FPGA. During hardware testing, the two 4-bit data vectors and the 3-bit operation selector are manipulated through the board's slider switches. Simultaneously, the computational outputs, along with a carefully designated fifth bit for overflow and error tracking, are monitored directly through the integrated LEDs.

Design Specifications

The developed ALU architecture is designed to process two primary 4-bit data vectors, utilizing an additional 3-bit selector signal to dictate the specific operation to be executed. The functionality is mapped to the following 3-bit selector codes:

- **000** - Unsigned Addition
- **001** - Signed Subtraction
- **010** - Bitwise OR
- **011** - Bitwise AND
- **100** - Logical Shift Left
- **101** - Bitwise XOR
- **110** - Logical Shift Right
- **111** - Signed Negation

To successfully reduce arithmetic overflow issues during addition, the final output size was expanded to incorporate a fifth bit. This supplementary bit is exclusively active and relevant during the addition and negation modes. When performing addition, this bit functions simply as the most significant bit of the final sum. Conversely, during signed

subtraction and negation, representing values outside the 4-bit two's complement limits (less than -8 or greater than +7) results in calculation errors. In such out-of-bounds scenarios, the fifth LED is illuminated to serve as a clear error flag.

The hardware constraints of the Basys3 FPGA directly dictated the 4-bit data limit. Because the device only supplies 16 slider switches, dedicating 3 switches for the operation selector leaves merely 13 available inputs. Consequently, scaling the design to an 8-bit ALU was not feasible within the board's physical parameters. Thus, the 4-bit scale represents the optimal maximum capacity for this specific FPGA. For the hardware mapping, the selector controls occupy the three leftmost switches, whereas the immediate four switches on the right correspond to input vector *A*, and the attached four switches map to vector *B*.

Regarding numerical interpretation, the unsigned format assigns bit weights as powers of two (1, 2, 4, etc.). In contrast, the signed format maintains these weights for the lower bits, while the most significant bit carries a negative weight. This standard method is known as two's complement, and it serves as the foundation for all signed calculations within this project. Given the 4-bit architecture, the highest negative value dictated by the MSB is -8. Because activating all other bits alongside the MSB yields a net value of -1, the highest representable positive integer is strictly +7. Any computation producing a mathematical result below -8 or above +7 naturally triggers an error state. Furthermore, sticking strictly to the lab directives, utilizing automated Vivado addition statements (e.g., assigning a sum directly using an arithmetic operator) is completely omitted from the VHDL code.

The specific operational details are detailed below:

1. **Addition:** The unit computes the sum of two unsigned 4-bit variables, *A* and *B*. By incorporating the fifth output bit, the system prevents standard overflow and can display numbers up to decimal 30.
2. **Subtraction:** In this state, vector *B* is subtracted from vector *A*, treating both as signed integers. Because subtracting a larger positive value from a negative one can exceed the 4-bit signed bounds, the fifth bit acts exclusively as a boundary error flag. For example, a mathematical result like -10 cannot be visualized.
3. **Logical OR, AND, XOR:** These states process the inputs *A* and *B* strictly as bitwise vectors. Corresponding bits from each input are passed through the respective logic gates. The output is a standard 4-bit vector routed straight to the board's LEDs.
4. **Shift Left and Shift Right:** Utilizing only a single 4-bit input, the system shifts the entire bit sequence one position toward the specified direction. The empty position is inherently populated with a logical '0'. Operationally, left-shifting mimics multiplication by 2, while right-shifting effectively acts as integer division by 2.
5. **Negation:** This mode also requires only one signed 4-bit input to compute its negative equivalent. The fifth output bit functions as an error flag, activating exclusively if an attempt is made to negate -8, which cannot be represented positively in 4 bits.

Methodology

The design process commenced with the foundational development of a single-bit full adder utilizing VHDL. Unlike a basic half adder, this component is designed to process both incoming and outgoing carry signals (C_{in} and C_{out}). The logical behavior of this unit is dictated by the boolean expressions: Sum = A ⊕ B ⊕ C_{in} and C_{out} = (A · B) + (C_{in}· (A⊕B)). The corresponding truth table that governs this logic is presented below.

A	**B**	C_{in}	**Sum**	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 1: Truth table of a one-bit full adder.

After verifying the 1-bit module, a complete 4-bit adder was constructed by cascading four of these units together, linking the C_{out} of one stage directly to the C_{in} of the subsequent stage. To ensure accurate baseline addition, the very first carry-in port was tied to a constant '0'. The final carry-out (the fifth bit) was kept conceptually distinct, as the outputs for the remaining ALU operations are strictly 4-bit wide.

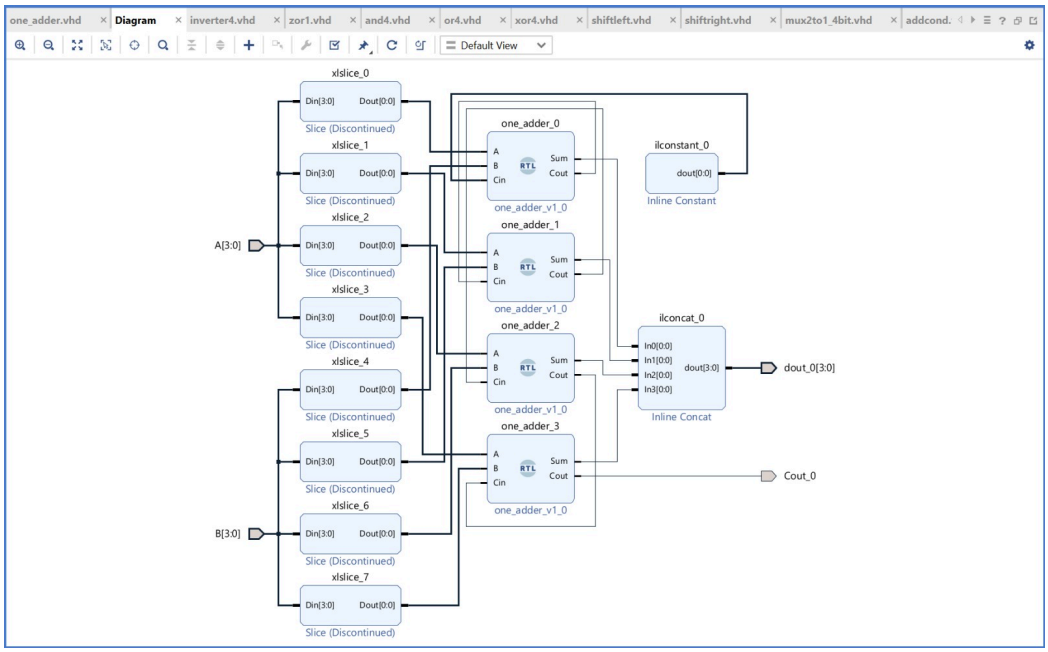


Figure 1: Block design of the 4-bit full adder

Building the 4-bit subtractor relied heavily on the existing adder architecture. By utilizing the fundamental arithmetic principle A – B = A + (−B), the subtraction operation was executed by adding the two's complement of B to A. To achieve this two's complement form, defined as −B = (not B) + 1,an inversion module was first created to flip all bits of vector B prior to routing it into the adder. Subsequently, rather than feeding a '0' to the initial C_{in} of the gradual adder, a constant '1' was supplied to automatically add the required integer.

A critical consideration during signed subtraction is mathematical overflow, which comes out when subtracting values with different signs yields an impossible 4-bit signed result. To systematically flag this, the carry signals corresponding to the Most Significant Bit (MSB) were analyzed. The error detection logic is dictated by the formula Error = $C_{in} \oplus C_{out}$. A dedicated 1-bit XOR gate was implemented to process these two signals and output the final error status.

A(msb)	**-B(msb)**	C_{in}	**Sum**	C_{out}	**Error**
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	1	0	0
0	1	1	0	1	0
1	0	0	1	0	0
1	0	1	0	1	0
1	1	0	0	1	1
1	1	1	1	1	0

Table 2: Truth table of the most significant full adder in subtraction, with Cin and Error.

For the logical operations (AND, OR, XOR), individual modules were scripted to accept two 4-bit vectors and perform parallel bitwise comparisons, outputting a 4-bit result directly. Because of their straightforward nature, these did not require complex internal routing and were integrated directly into the top-level ALU block diagram. Similarly, the shift functions (Left and Right) were implemented by realigning the input bits one position over and hardwiring a '0' into the newly vacated bit slot. These blocks were also appended directly to the main system. The standalone negation operation leveraged the previously built 4-bit inverter alongside the full adder to calculate the two's complement of a single input by inverting it and adding a constant binary 0001.

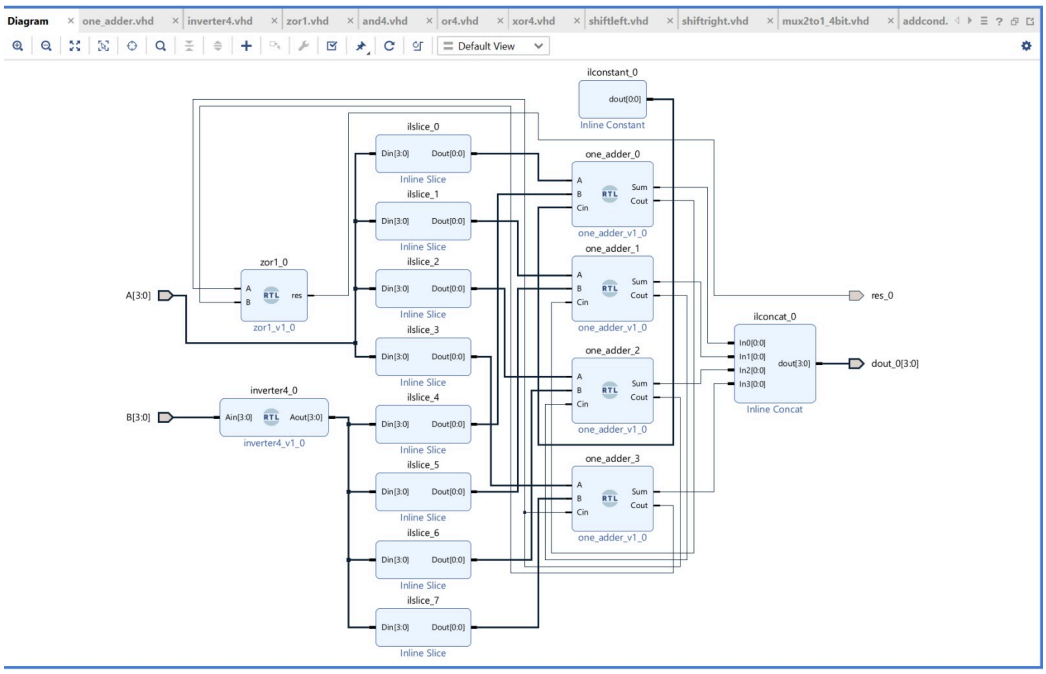


Figure 2: Block design of the subtractor

To effectively manage the execution of eight distinct commands, a 3-bit selector mechanism was required. This logic routing was achieved by constructing an 8-to-1 multiplexer (MUX) to funnel the correct operation to the board's LEDs. The foundational element of this router was a standard 2-to-1 MUX, governed by the boolean equation: Res = (A_1 and S) or (A_0 and not S). Note that while the theoretical logic evaluates 1-bit routing, the implemented VHDL version processes entire 4-bit vectors simultaneously by broadcasting the selection signal across all bits.

S	A_0	A_1	**Res**
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Table 3: Truth Table of a 1-bit 2 to 1 mux.

We constructed the 8-to-1 multiplexer by connecting seven individual 2-to-1 MUX blocks in a hierarchical structure. The design filters the input signals through three separate stages. In the first stage, four MUXes are controlled by S_0 to select four

intermediate signals. These signals then enter the second stage, where two MUXes are managed by S_1. Finally, S_2 dictates the last selection to determine the overall output. This tiered approach ensures that every ALU operation is mapped to a unique 3-bit selection vector.

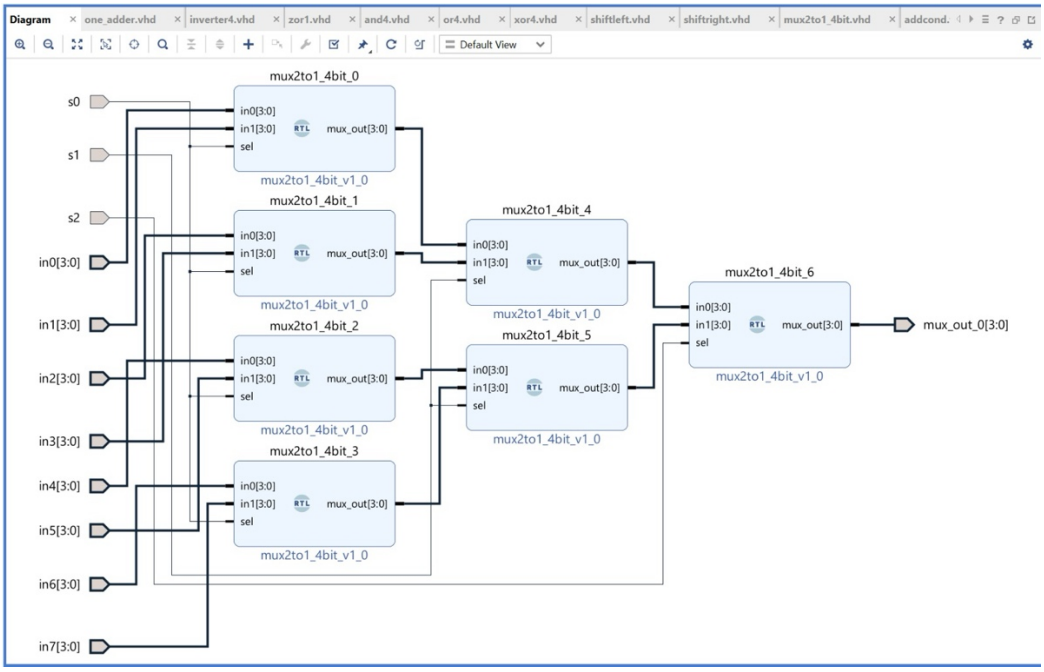


Figure 3: Block design of 4-bit 8 to 1 multiplexer

Lastly, we designed a distinct module to control the fifth LED output. This extra bit is designed to turn on in only three specific cases: to represent the decimal value 16 during unsigned addition, and to act as an error indicator for overflow during signed subtraction and negation. The module functions as a conditional checker; if the ALU is not running one of these three operations, it simply assigns a '0' to the fifth bit. However, if one of those specific modes is active, the module evaluates the relevant carry-out and carry-in signals of the adders. It also checks for the unique error condition in negation, which happens when trying to negate 1000 in binary. The output signal of this control block is defined by the following VHDL assignment:

```
out0 <= (is_addition and in0) or (is_negation and negation_error) or (is_subtraction and in1);
```

This single conditional bit is then concatenated with the 4-bit multiplexer result to generate the comprehensive 5-bit vector sent to the hardware LEDs.

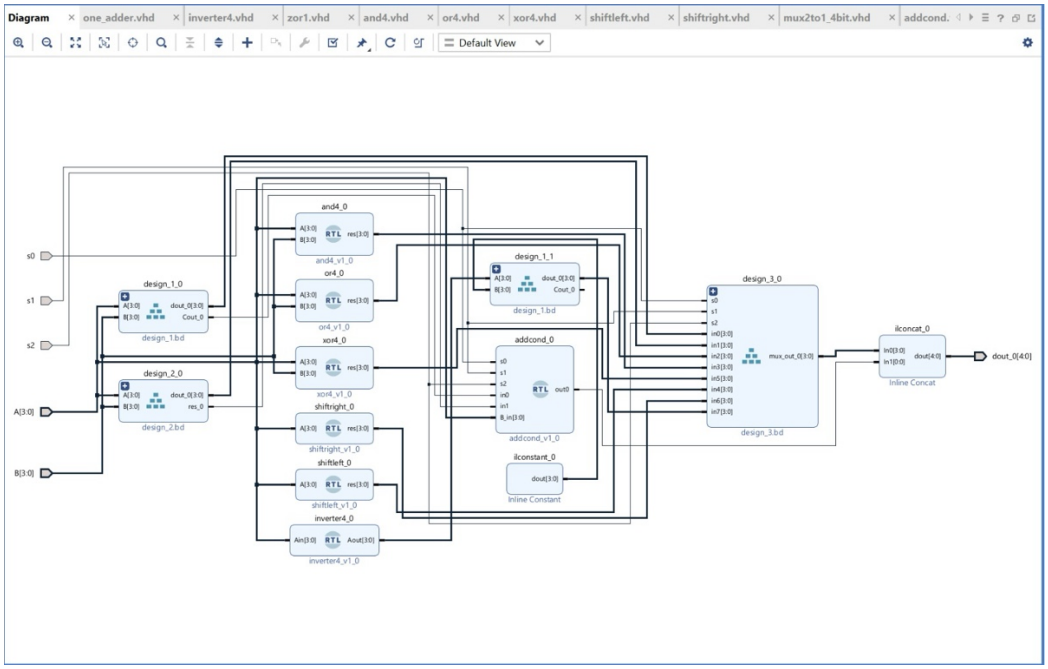


Figure 4: The final design of ALU

To verify our digital design, we wrote a testbench in VHDL. We used nested `for` loops to go through all the possible combinations for the inputs A and B, as well as the 3-bit selector. Once we saw the correct results in the simulation, we prepared the constraint file. This file linked our ALU's input and output ports to the actual switches and LEDs on the Basys3 board. As the last step, we generated the bitstream and loaded it onto the FPGA.

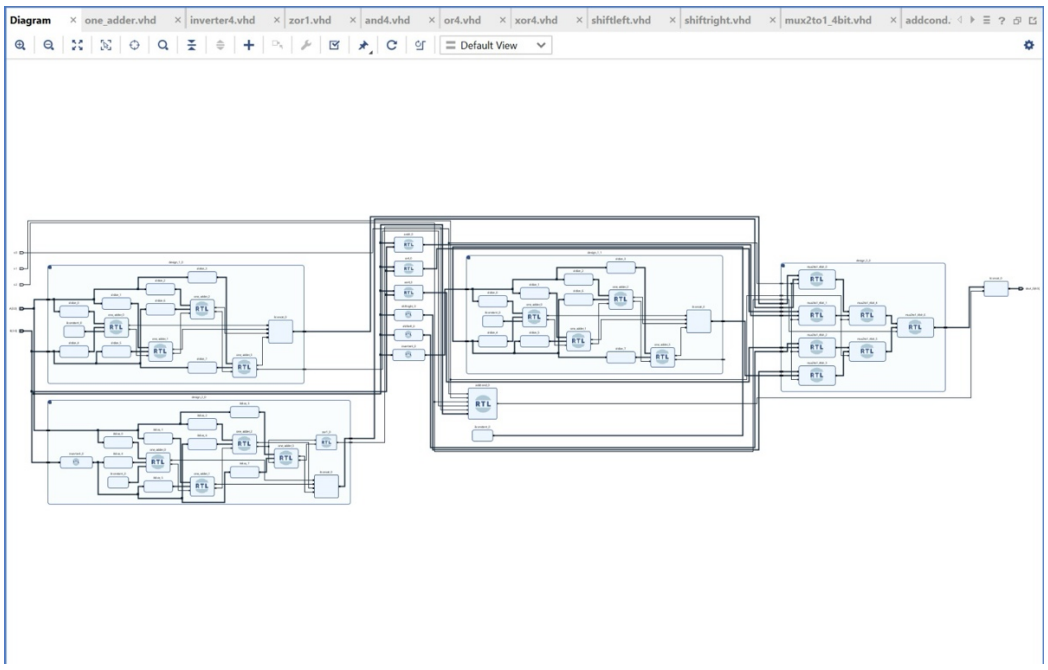


Figure 5: Elaborated design of the final ALU

Results

Before implementing the design on the Basys3 FPGA board, we tested our digital architecture using a VHDL testbench. The simulation waveforms matched our expected theoretical results perfectly. For example, in the unsigned addition mode, giving the decimal inputs 3 and 9 correctly produced the decimal sum of 12.

Our system has 211 possible input combinations in total. Because of this large amount of data, we could not easily fit all of them into Vivado's standard 1000-nanosecond simulation window, even with the wait time set to 1 nanosecond. As a result, the output waveforms look very thin on the screen, sometimes forming a continuous solid block. However, despite this visual limitation, all the simulated inputs generated the correct mathematical outputs.

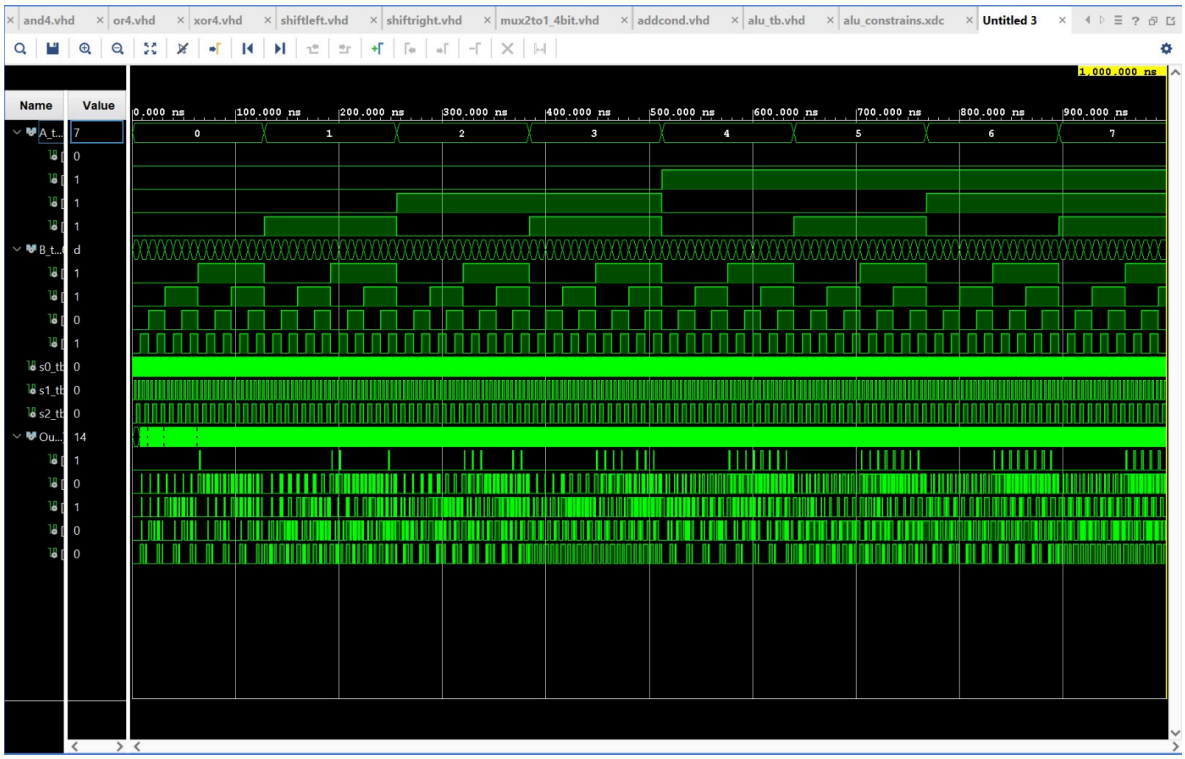


Figure 6: Simulation results of the testbench code

After the simulation was done, we wrote the constraint file to map the inputs and outputs, and generated the bitstream. We programmed the Basys3 board, and the physical device worked exactly as we expected from the simulations.

We tested all eight operations on the board to make sure everything was correct.

For the addition mode, the LEDs showed the correct 5-bit unsigned results, like adding 12 and 10 to get 22. In subtraction mode, normal calculations such as 5 - 2 = 3 worked perfectly. Also, when we tried an out-of-range operation, like subtracting 7 from -6, the fifth error LED lit up as intended. The bitwise logic operations (OR, AND, XOR) and the shiftleft/shiftright modes processed the binary inputs correctly without any issues. Lastly, the negation mode successfully inverted the signed numbers , and the error LED turned on when we tried to negate -8.

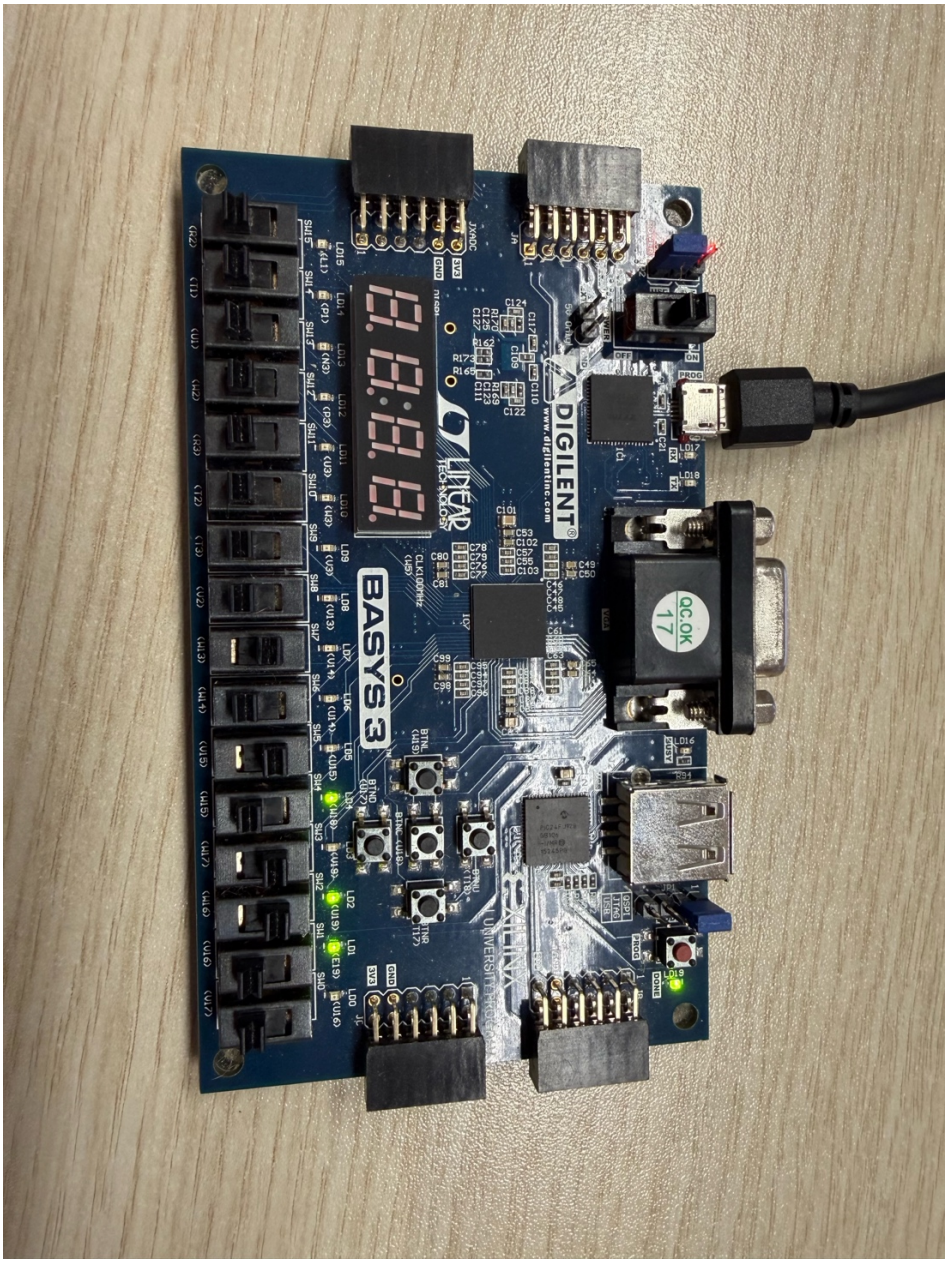


Figure 7: Basys3 in addition mode, showing 12 + 10 = 22 in decimal, with 4-bit inputs and 5-bit output all unsigned

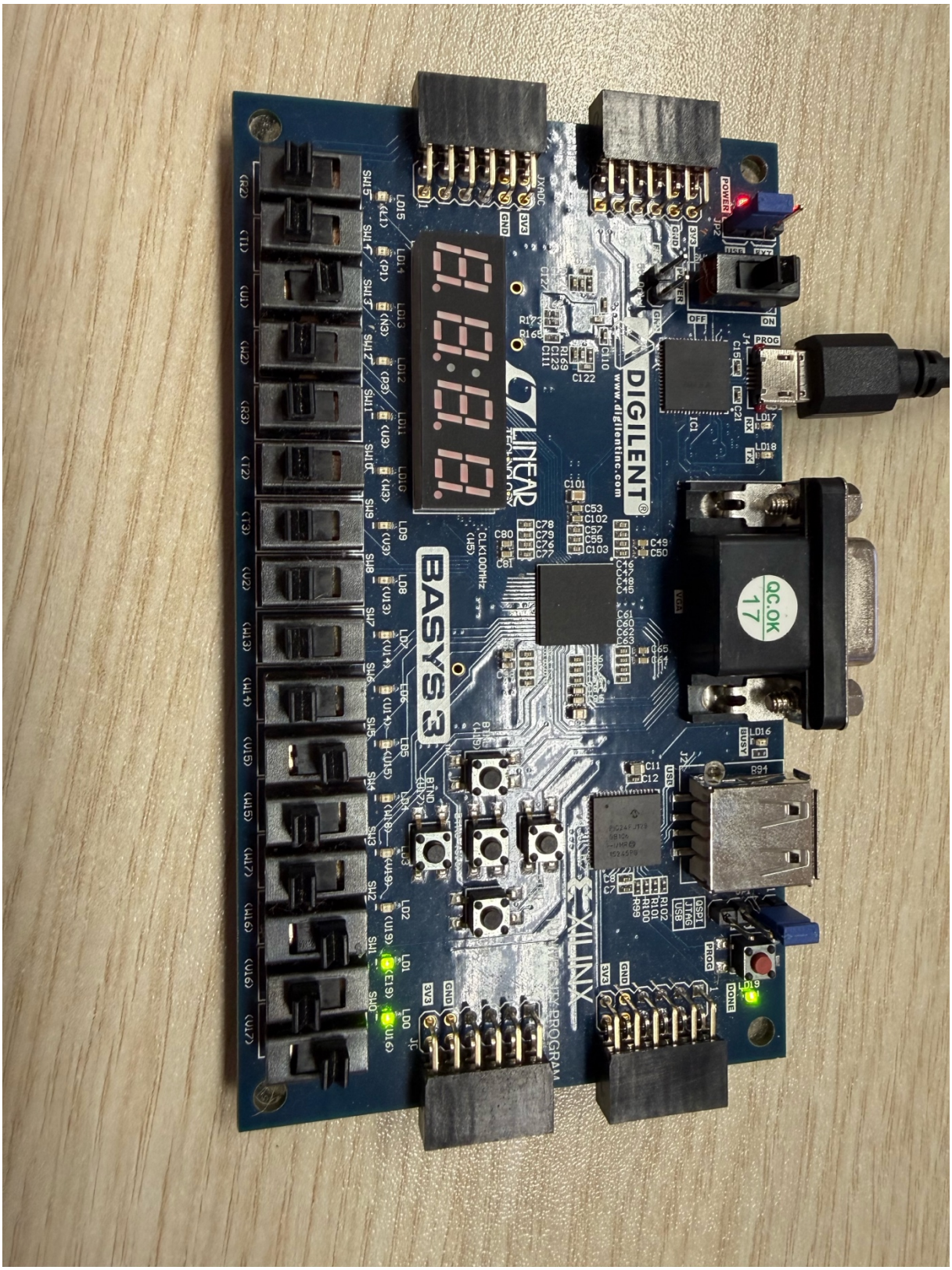


Figure 8: Basys3 in subtracting mode, showing 5 - 2 = 3 in decimal. All 4-bit signed

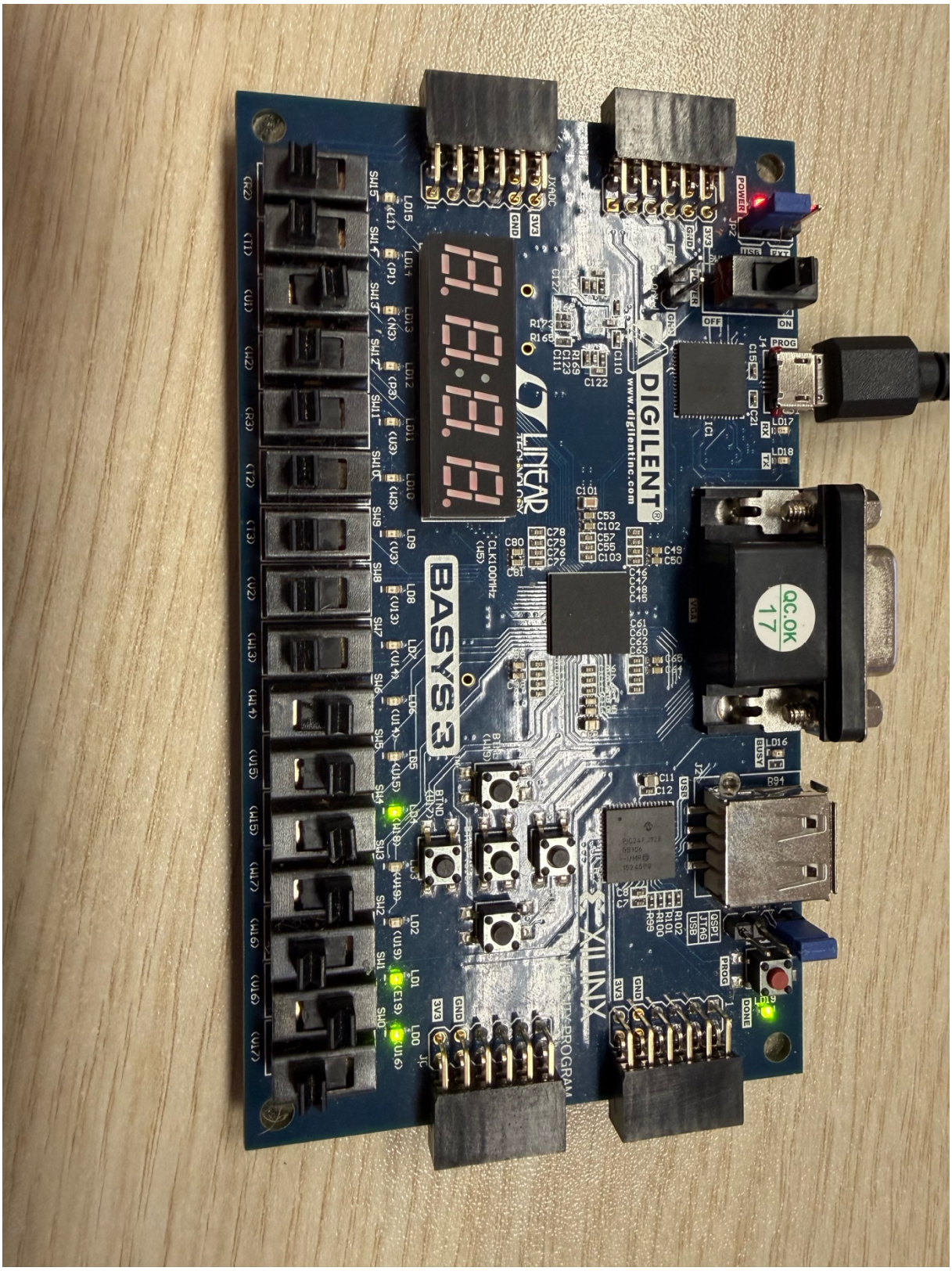


Figure 9: Basys3 in subtraction mode, showing -6 - (+7) ≠ +3 with the error indicator LED lit up

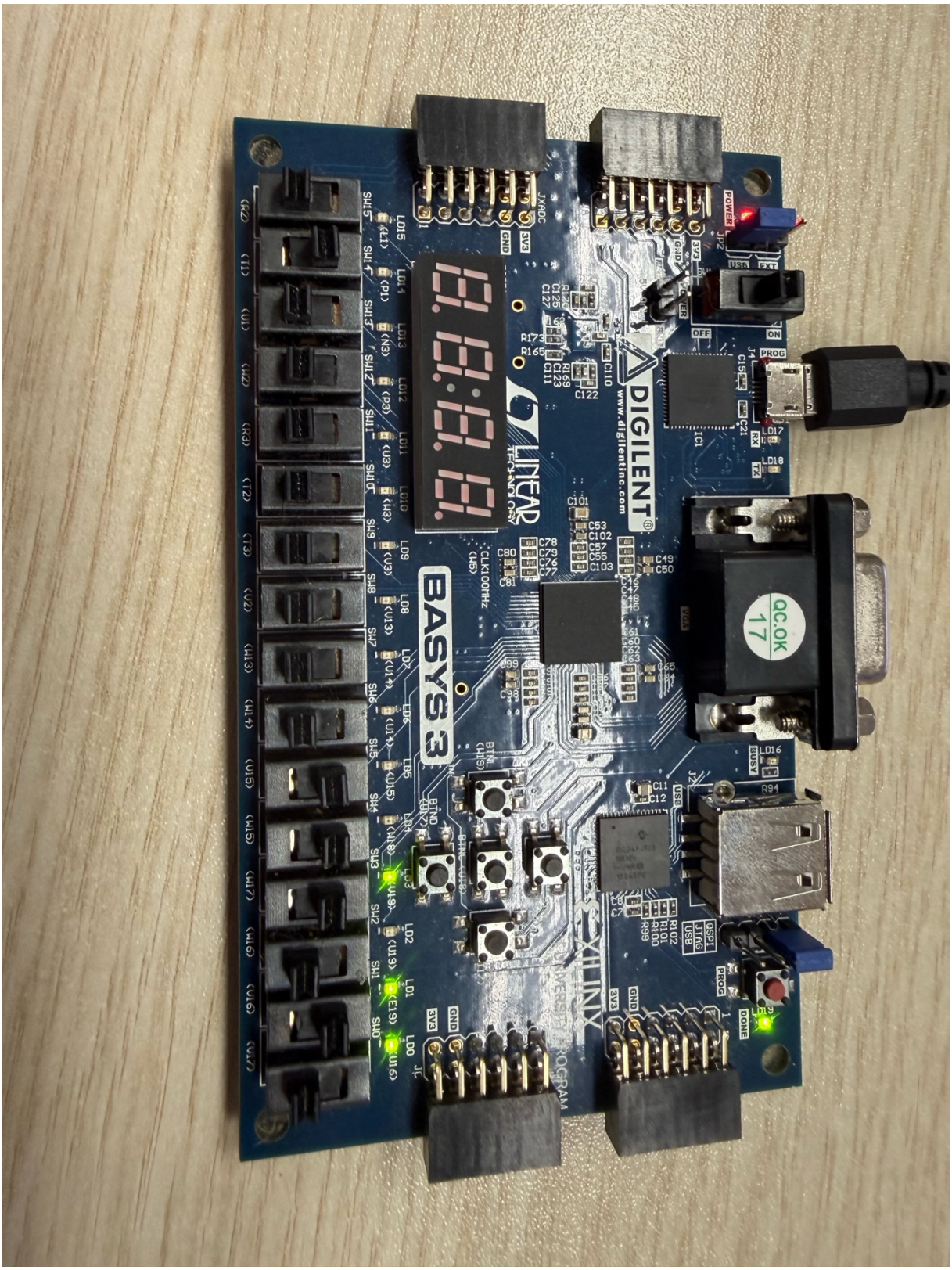


Figure 10: Basys3 in OR mode, inputs 1010 and 0011 resulting in 1011

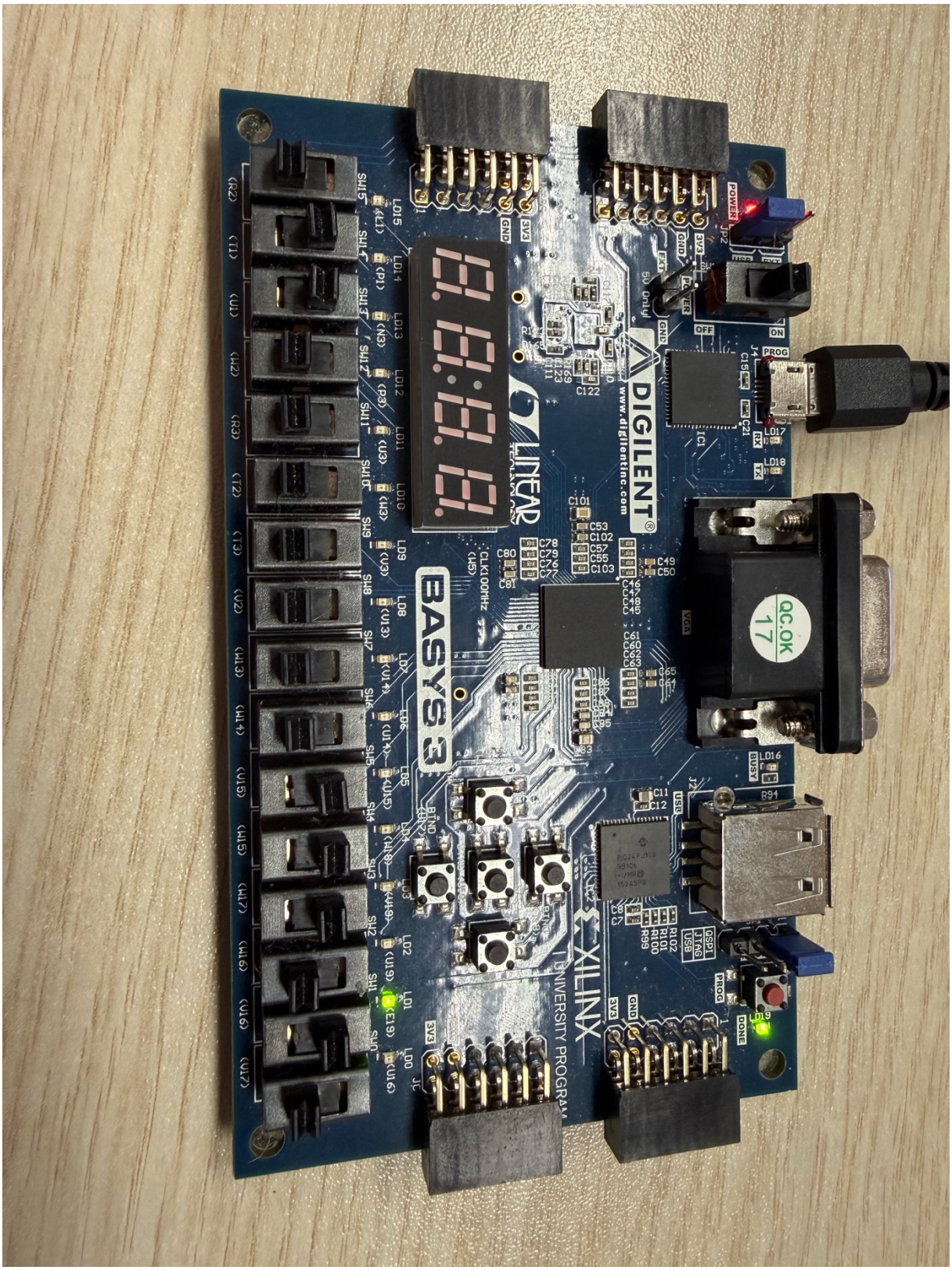


Figure 11: Basys3 in AND mode, inputs 1010 and 1011 resulting in 0010

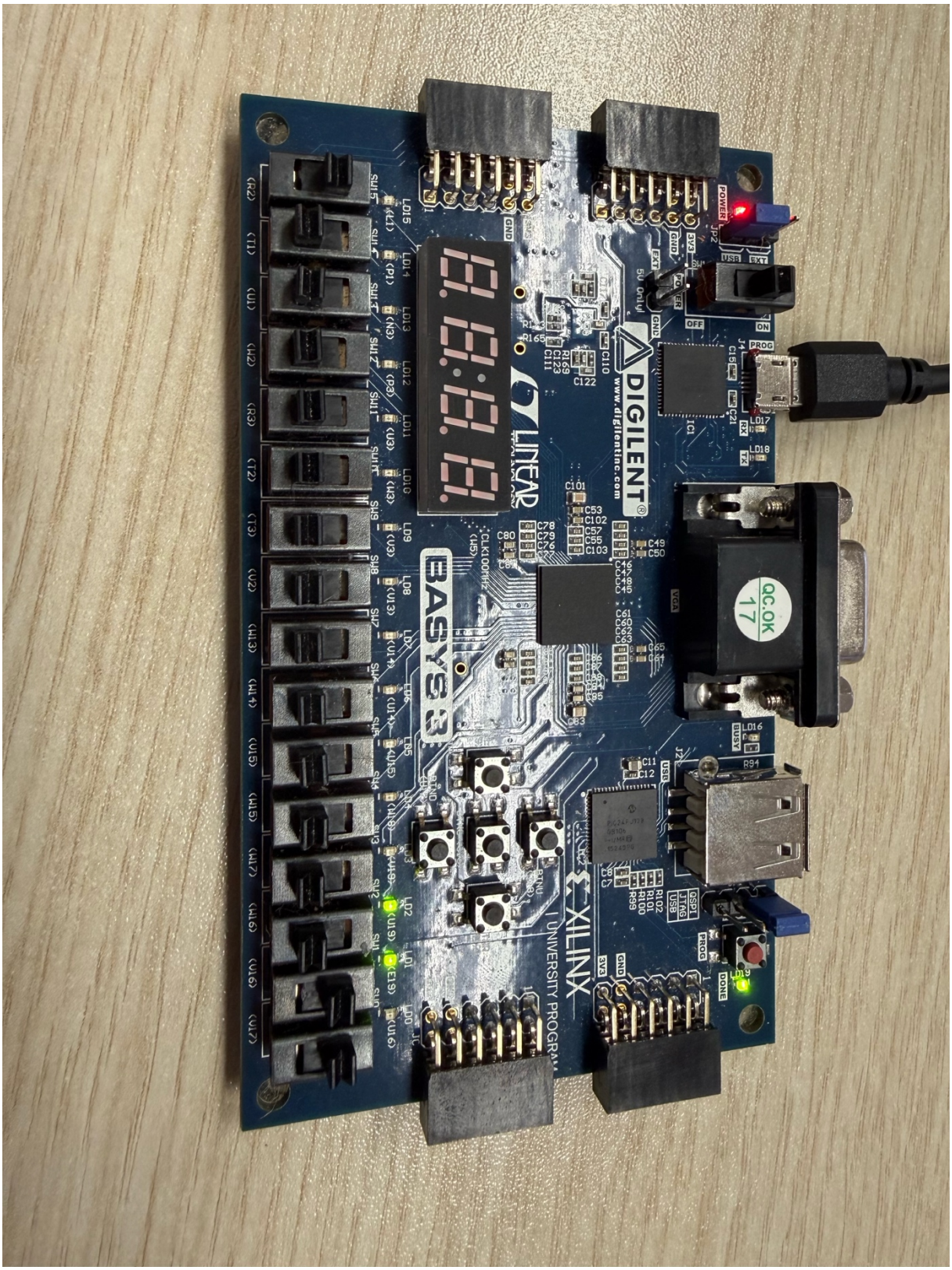


Figure 12: Basys3 in shiftleft mode, input 0011 resulting in 0110

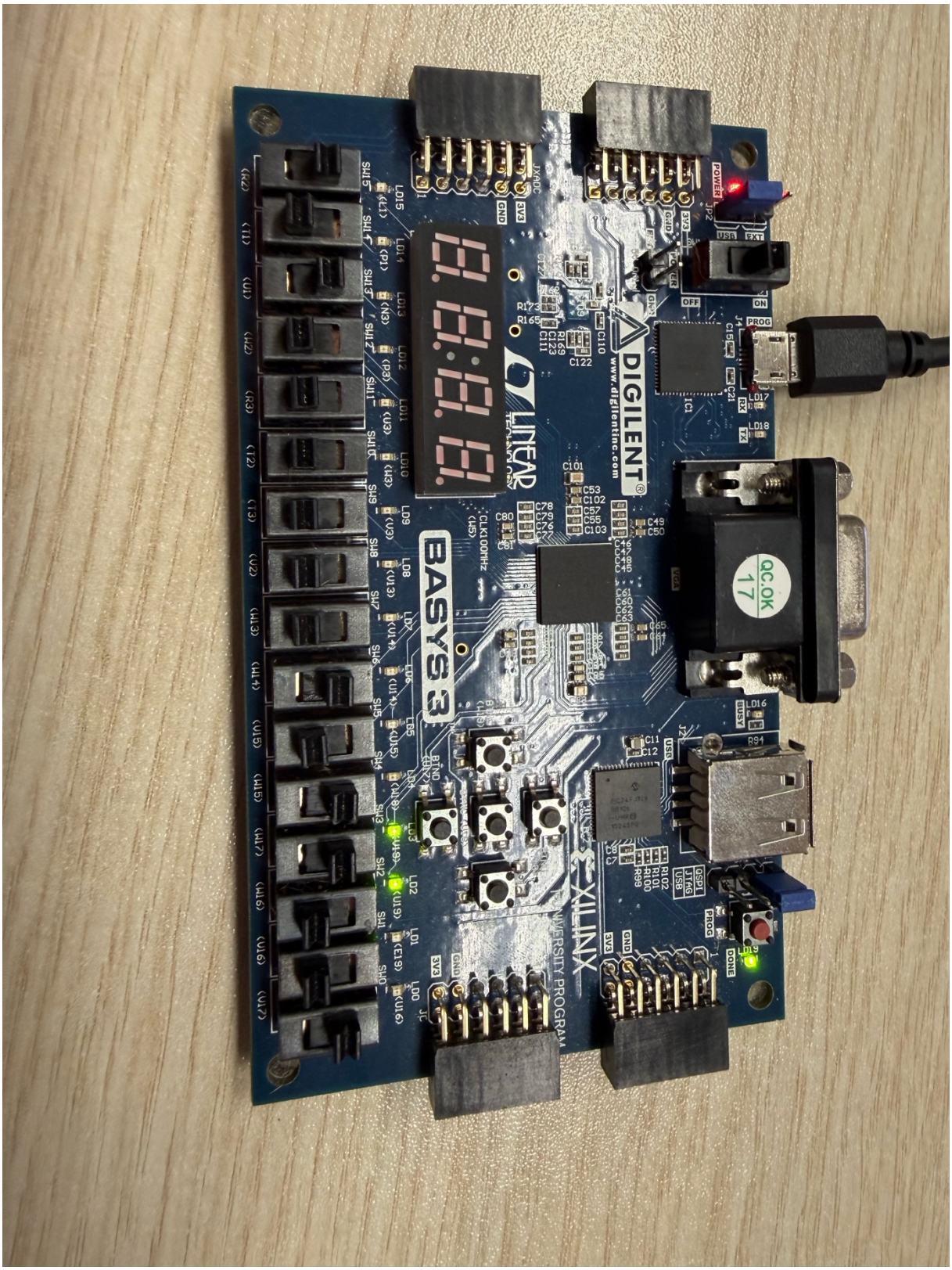


Figure 13: Basys3 in XOR mode, inputs 0101 and 1001 resulting in 1100

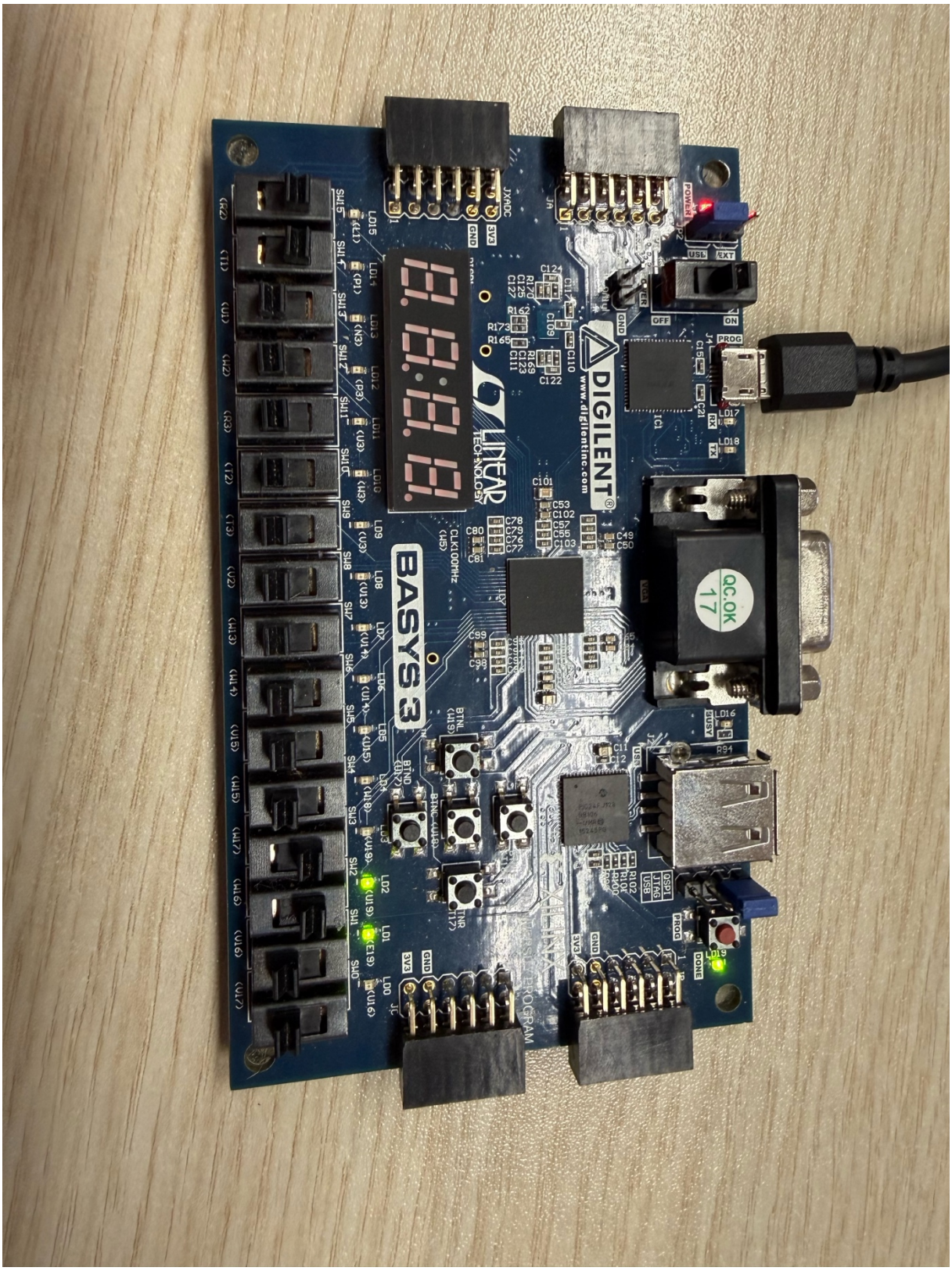


Figure 14: Basys3 in shiftright mode, the input 1100 resulting in 0110

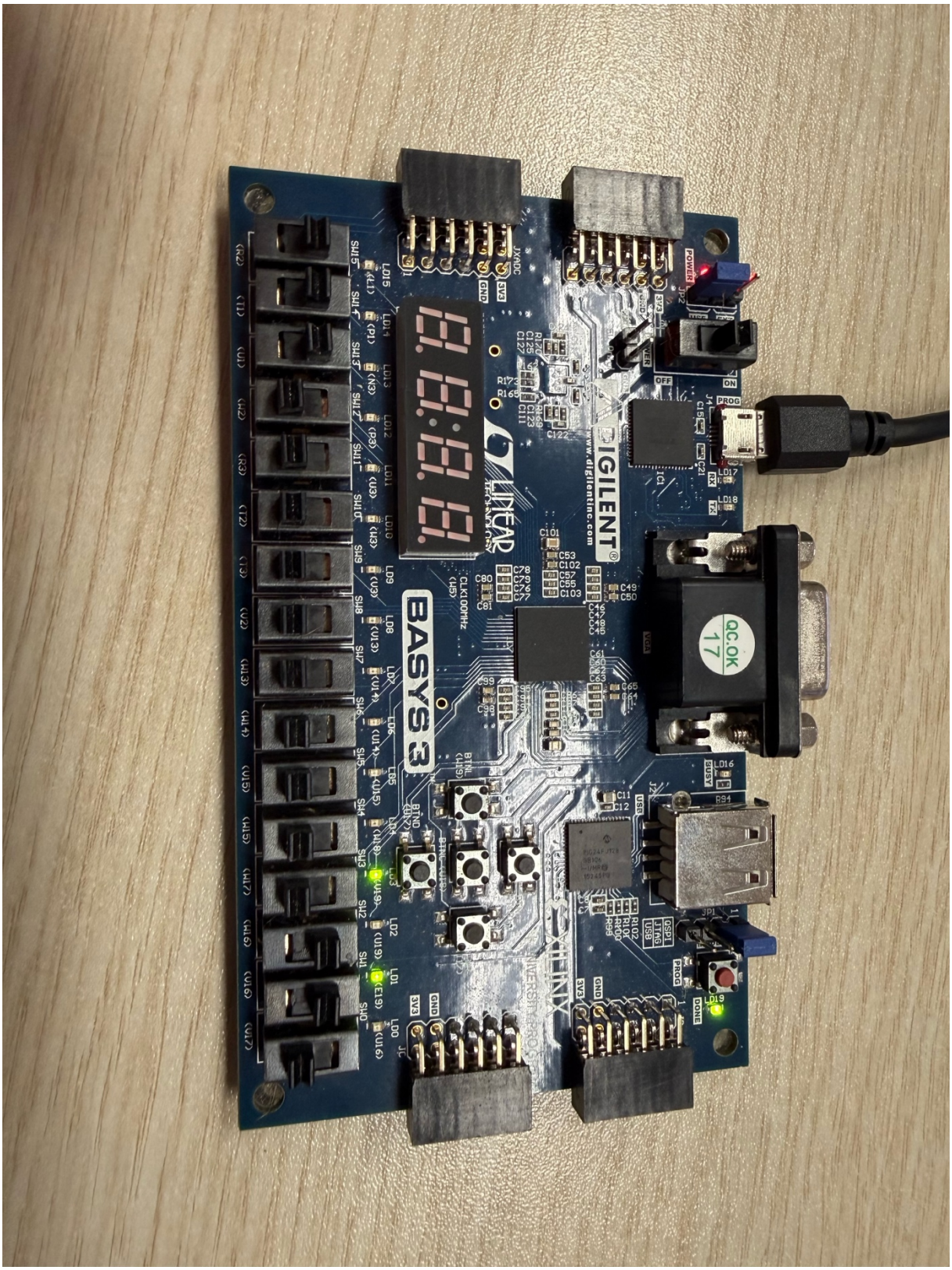


Figure 15: Basys3 in negation mode, input +6 in decimal resulting in -6 in decimal

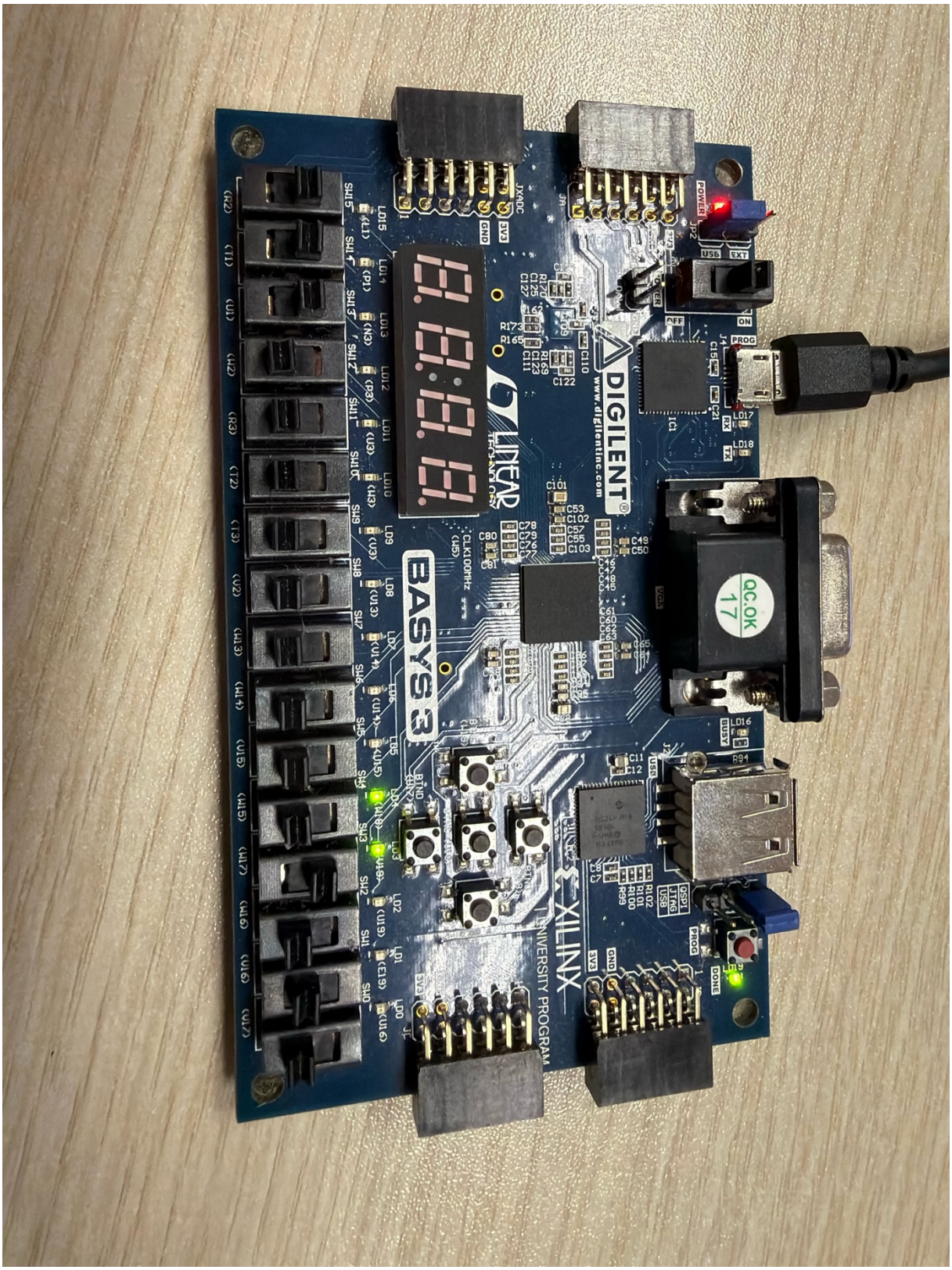


Figure 16: Basys3 in negation mode, input -8 resulting in -(-8) ≠ -8, as the error indicator LED is lit up

Conclusion

In this lab, a fully functional 4-bit Arithmetic Logic Unit (ALU) was successfully designed, simulated, and implemented on the Basys 3 FPGA board. The project demonstrated that complex digital processing units can be built using basic logic gates and a structural design approach, without relying on built-in arithmetic libraries. Creating 1-bit full adders and basic logic gates as modular blocks, and then combining them with an 8-to-1 multiplexer, highlighted the practical importance of hierarchical hardware design.

A key outcome of this experiment was the observation of signed two's complement arithmetic and its physical limitations. By expanding the ALU output to five bits, the design successfully handled standard addition carry-outs and included a dedicated hardware error flag. This flag effectively indicated overflow errors, such as the mathematical impossibility of negating -8 within a 4-bit signed system.

Furthermore, the verification process was essential for the success of the design. Using a VHDL testbench to simulate the majority of the 2,048 possible input combinations helped identify and correct early routing issues in the Vivado block design. Overall, the correct operation of the ALU on the FPGA board fulfilled all the lab's objectives, effectively connecting theoretical Boolean algebra with applied digital electronics.

Appendix

VHDL code fort he one bit full adder

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity one_adder is
Port(
   A: in std_logic;
   B: in std_logic;
   Cin: in std_logic;
   Sum: out std_logic;
   Cout: out std_logic
);
end one_adder;
```

```
architecture Behavioral of one_adder is
begin
   Sum <= A xor B xor Cin;
   Cout <= (A and B) or (Cin and (A xor B));
end Behavioral;
```

VHDL code for the 4-bit inverter

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity inverter4 is
Port(
   Ain: in std_logic_vector (3 downto 0);
   Aout: out std_logic_vector (3 downto 0)
);
end inverter4;

architecture Behavioral of inverter4 is
begin
   Aout <= not Ain;
end Behavioral;
```

VHDL code for the AND module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity and4 is
```

```
Port(
   A: in std_logic_vector (3 downto 0);
   B: in std_logic_vector (3 downto 0);
   res: out std_logic_vector (3 downto 0)
);
end and4;

architecture Behavioral of and4 is
begin
   res <= A and B;
end Behavioral;
```

VHDL code for the OR module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity or4 is
Port(
   A: in std_logic_vector (3 downto 0);
   B: in std_logic_vector (3 downto 0);
   res: out std_logic_vector (3 downto 0)
);
end or4;

architecture Behavioral of or4 is
begin
   res <= A or B;
end Behavioral;
```

VHDL code fort he XOR module, 4-bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity xor4 is
Port(
   A: in std_logic_vector (3 downto 0);
   B: in std_logic_vector (3 downto 0);
   res: out std_logic_vector (3 downto 0)
);
end xor4;

architecture Behavioral of xor4 is
begin
   res <= A xor B;
end Behavioral;
```

VHDL code for the XOR module, 1-bit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity zor1 is
Port(
   A: in std_logic;
   B: in std_logic;
   res: out std_logic
);
```

```
end zor1;

architecture Behavioral of zor1 is
begin
   res <= A xor B;
end Behavioral;
```

VHDL code for the 4-bit shiftleft module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shiftleft is
Port(
   A: in std_logic_vector (3 downto 0);
   res: out std_logic_vector (3 downto 0)
);
end shiftleft;

architecture Behavioral of shiftleft is
begin
   res <= A(2 downto 0) & '0';
end Behavioral;
```

VHDL code for the 4-bit shiftright module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shiftright is
```

```
Port(
   A: in std_logic_vector (3 downto 0);
   res: out std_logic_vector (3 downto 0)
);
end shiftright;

architecture Behavioral of shiftright is
begin
   res <= '0' & A(3 downto 1);
end Behavioral;
```

VHDL code for the 4-bit 2 to 1 mux module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux2to1_4bit is
Port(
   in0: in std_logic_vector (3 downto 0);
   in1: in std_logic_vector (3 downto 0);
   sel: in std_logic;
   mux_out: out std_logic_vector (3 downto 0)
);
end mux2to1_4bit;

architecture Behavioral of mux2to1_4bit is
   signal sel_vector: std_logic_vector (3 downto 0);
   signal not_sel_vector: std_logic_vector (3 downto 0);
begin
```

```
  sel_vector <= (others => sel);
  not_sel_vector <= not sel_vector;
  mux_out <= (in0 and not_sel_vector) or (in1 and sel_vector);
end Behavioral;
```

VHDL code for the error condition module

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity addcond is
Port(
  s0: in std_logic;
  s1: in std_logic;
  s2: in std_logic;
  in0: in std_logic;
  in1: in std_logic;
  B_in: in std_logic_vector (3 downto 0);
  out0: out std_logic
);
end addcond;

architecture Behavioral of addcond is
  signal is_addition: std_logic;
  signal is_negation: std_logic;
  signal negation_error: std_logic;
  signal is_subtraction: std_logic;
begin
  is_addition <= not s0 and not s1 and not s2;
```

```
  is_negation <= s0 and s1 and s2;
  is_subtraction <= not s2 and not s1 and s0;
  negation_error <= B_in(3) and not B_in(2) and not B_in(1) and not B_in(0);
  out0 <= (is_addition and in0) or (is_negation and negation_error) or (is_subtraction and
in1);
end Behavioral;
```

VHDL code for the testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu_tb is
end alu_tb;

architecture behavior of alu_tb is
  component alu_design_wrapper
  port(
    A : in std_logic_vector(3 downto 0);
    B : in std_logic_vector(3 downto 0);
    s0 : in std_logic;
    s1 : in std_logic;
    s2 : in std_logic;
    dout_0: out std_logic_vector(4 downto 0)
  );
  end component;

  signal A_tb: std_logic_vector(3 downto 0) := (others => '0');
```

```
  signal B_tb: std_logic_vector(3 downto 0) := (others => '0');
  signal s0_tb: std_logic := '0';
  signal s1_tb: std_logic := '0';
  signal s2_tb: std_logic := '0';
  signal Out_tb: std_logic_vector(4 downto 0);
begin
  uut: alu_design_wrapper port map (
    A => A_tb,
    B => B_tb,
    s0 => s0_tb,
    s1 => s1_tb,
    s2 => s2_tb,
    dout_0 => Out_tb
  );

  stim_proc: process
    variable sel_var: unsigned(2 downto 0);
  begin
    for a_int in 0 to 15 loop
      A_tb <= std_logic_vector(to_unsigned(a_int, 4));
      for b_int in 0 to 15 loop
        B_tb <= std_logic_vector(to_unsigned(b_int, 4));
        for sel_int in 0 to 7 loop
          sel_var := to_unsigned(sel_int, 3);
          s2_tb <= sel_var(2);
          s1_tb <= sel_var(1);
          s0_tb <= sel_var(0);
          wait for 1 ns;
```

```
            end loop;
          end loop;
        end loop;
    end process;
end behavior;
```

VHDL code for the constrain file

```
set_property PACKAGE_PIN V17 [get_ports {A[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[0]}]

set_property PACKAGE_PIN V16 [get_ports {A[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[1]}]

set_property PACKAGE_PIN W16 [get_ports {A[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[2]}]

set_property PACKAGE_PIN W17 [get_ports {A[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {A[3]}]

set_property PACKAGE_PIN W15 [get_ports {B[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[0]}]

set_property PACKAGE_PIN V15 [get_ports {B[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[1]}]

set_property PACKAGE_PIN W14 [get_ports {B[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[2]}]
```

```
set_property PACKAGE_PIN W13 [get_ports {B[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {B[3]}]

set_property PACKAGE_PIN U1 [get_ports s0]
set_property IOSTANDARD LVCMOS33 [get_ports s0]

set_property PACKAGE_PIN T1 [get_ports s1]
set_property IOSTANDARD LVCMOS33 [get_ports s1]

set_property PACKAGE_PIN R2 [get_ports s2]
set_property IOSTANDARD LVCMOS33 [get_ports s2]

set_property PACKAGE_PIN U16 [get_ports {dout_0[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {dout_0[0]}]

set_property PACKAGE_PIN E19 [get_ports {dout_0[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {dout_0[1]}]

set_property PACKAGE_PIN U19 [get_ports {dout_0[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {dout_0[2]}]

set_property PACKAGE_PIN V19 [get_ports {dout_0[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {dout_0[3]}]

set_property PACKAGE_PIN W18 [get_ports {dout_0[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {dout_0[4]}]
```

References

[1] Bilkent University Department of Electrical and Electronics Engineering, "EEE 102: Digital Design Laboratory Manual - Lab 4: Arithmetic Logic Unit," Ankara, Turkey, Spring 2026.

[2] Digilent Inc., "Basys 3 FPGA Board Reference Manual," Rev. C. [Online]. Available: https://digilent.com/reference/programmable-logic/basys-3/reference-manual

[3] Xilinx Inc., "Vivado Design Suite User Guide: Logic Simulation (UG900)," San Jose, CA. [Online]. Available: https://docs.xilinx.com/r/en-US/ug900-vivado-logic-simulation

[4] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with VHDL Design*, 3rd ed. New York, NY: McGraw-Hill Education, 2008.

[5] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6th ed. Pearson, 2012.

[6] R. Merrick, "VHDL Tutorials and Examples: Adders and Multiplexers," *Nandland*. [Online]. Available: https://nandland.com/vhdl-tutorials/

[7] J. J. Jensen, "How to create a basic testbench in VHDL," *VHDLwhiz*. [Online]. Available: https://vhdlwhiz.com/basic-testbench/

[8] A. Yılmaz, "Vivado Block Design and FPGA Implementation Guide," Bilkent University EEE 102 Supplementary Laboratory Notes, Spring 2026.

[9] K. Demir, "Basys 3 Master XDC Constraint Mapping Tutorial," Bilkent University EEE 102 Course Materials, Spring 2026.

[10] IEEE, "IEEE Standard VHDL Language Reference Manual," IEEE Std 1076-2008, Jan. 2009.